Software Metrics in Software Engineering

1. Software Metrics :

Software metrics in software engineering are the standards for estimating the quality, **progress**, and health of **software development** activity. A software metric is a quantifiable or countable assessment of the qualities of software.

2. Characteristics of Software Metrics:

Software metrics in software engineering should possess the following characteristics:

- **Quantitative**: Metrics must be quantitative in order to be helpful. It means that metrics can be **stated numerically**.
- **Understandable**: Metric computation should be simple to **grasp**, and the method for **computing** them should be well explained.
- **Applicability**: Metrics should be applied during the early stages of **software development**.
- **Repeatable**: When measured regularly and **consistently in nature**, the metric values should be the same.
- **Economical**: Metric computation should be **cost-effective**.
- **Language agnostic**: Metrics should be **language-independent**, meaning their computation should not depend on any **programming language**.

3. Need for Software Metrics

Software metrics in software engineering can be used for a variety of purposes, including analyzing software performance, planning work items, estimating productivity, and so on. For example, you can use software metrics to **monitor performance**, plan upcoming work tasks, **track productivity**, and better regulate the production process throughout project management if you can view distinct statistics and trends as production takes place. You can also use software metrics in conjunction with management functions to simplify your projects by building more **efficient procedures**, creating software maintenance plans, and keeping production teams updated on issues that need to be resolved.

Software metrics are needed for various reasons, including the following:

- By preventing faults, it is possible to save money.
- Aids in the **improvement** of overall project planning
- Aids in the **advancement** of overall project planning
- Encourages **eagerness** to improve processes
- Aids in a more thorough analysis of the **dangers involved**.
- Aids in studying **indicators** at each testing stage to increase problem eradication efficiency.
- Increases the **ROI** of test automation over time
- Improves the relationship between **testing coverage**, **risks**, and **system complexities**.

4. Classification of Software Metrics:

There are three types of software metrics in software engineering: **product metrics**, **process metrics**, and **project metrics**. Product metrics are used to define product attributes such as size, **design features**, **complexity**, performance, **level of quality**, and so on. In contrast, process metrics are used to optimize software development and maintenance. Project metrics, such as the number of software developers, **cost**, and so on, indicate the project's characteristics and performance.



SCALER Topics

a). Product Metrics:
Product metrics represent product attributes such as size, **complexity**, **design features**, **performance**, and **quality level**. A working product is created at the end of each successive phase of the **software development process**. At any step of development, a product can be measured. Metrics are built for these items to determine whether a product is being developed in accordance with user requirements. If a product fails to satisfy **consumer expectations**, the relevant steps are made in the appropriate phase. Product metrics assist software engineers in detecting and correcting possible issues before they cause catastrophic failures.

b). Process Metrics:
To improve any process, it is important to measure its defined properties, establish a set of meaningful metrics based on these attributes, and then utilize these metrics to generate indicators to develop a process improvement strategy. Software engineers can measure the performance of a software process that is executed using the process as a framework by utilizing **software process metrics**. The process is at the center of the triangle that connects three aspects **(product, people, and technology)** that significantly impact software quality and organizational performance. People's competence and motivation, the complexity of the product, and the level of technology utilized in software development all significantly **impact quality** and **team performance**.

c). Project Metrics:
Project metrics explain the characteristics and execution of a project. Examples include the number of software developers, the staffing pattern throughout the **software's life cycle**, **cost**, **schedule**, and **productivity**. The project manager monitors the project's progress using measures known as project metrics. Various metrics, such as time, cost, and so on, are collected using data from previous projects and utilised as an estimate for the new initiative. The project manager monitors the project's progress on a **regular basis**, and **effort**, **time**, and cost are compared to the initial effort, time, and cost. These indicators can help lower development costs, efforts, hazards, and time. The project's quality can also be improved. With an improvement in quality, there is a decrease in the number of errors, time, cost, and so on.

5. Types of Software Metrics:
- **Internal metrics:** Internal metrics are used to measure properties that are deemed more important to a software developer than the users. Lines of Code **(LOC)** is one example.
- **External metrics:** External metrics are used to measure features that are deemed more important to the user than the software developers, such as **portability**, **reliability**, **functionality**, **usability**, and so on.
- **Hybrid Metrics:** Metrics that **mix product**, **process**, and resource metrics are known as **hybrid metrics**. Cost per FP is an example, where FP stands for Function Point Metric.

6. Advantages of Software Metrics:
- The software system design technique can be analyzed in comparison to other designs.
- Using software metrics, the properties of various **programming languages** can be explored for study and comparison.
- Software metrics in **software engineering** can be used to create software quality requirements.
- Software system compliance with **requirements** and standards can be verified.
- It is possible to infer the effort required to **design** and **develop** software systems.
- The code's **complexity** can be calculated.
- It is possible to decide whether or not to divide a **complex module** into smaller ones.
- It is possible to direct the best use of **resource managers**.
- It is possible to make design **trade-offs** and compare maintenance and software development expenses.

7. Disadvantages of Software Metrics:

- It isn't easy to apply measurements in all situations. In some **circumstances**, it is difficult and costly.
- It is challenging to validate historical or empirical evidence used for **verification** and **argumentation**.
- Although software products can be managed, technical staff performance cannot be measured using **software metrics**.
- The available tools and working environment are utilized to define and **derive software metrics**, and there is no standard for doing so.
- Certain variables are estimated using **predictive models** and are not always known.

8. Conclusion:

- Measuring the progress of software development teams can be a critical step toward increasing efficiency and **productivity**.
- Software metrics can be used to track a range of **software development aspects**.
- Metrics are used for measurement. Processes are measured using process metrics, products are measured using **product metrics**, and projects are measured using project metrics.
- Process metrics evaluate the effectiveness and quality of software processes, as well as the **process's maturity**, the effort required, and the success of defect removal during development, among other things.
- Product metrics are used to assess the quality of work **generated** during various stages of **software development**.
- Project metrics depict project **features and execution**.

Cyclomatic Complexity

Cyclomatic Complexity is a product metric used to measure the control complexity of a program. It is the quantitative measure of the number of linearly independent paths through the source code.

Cyclomatic complexity is a software metric used to measure the complexity of a program. Thomas J. McCabe developed this metric in 1976.McCabe interprets a computer program as a set of a strongly connected directed graph. Nodes represent parts of the source code having no branches and arcs represent possible control flow transfers during program execution. The notion of program graph has been used for this measure, and it is used to measure and control the number of paths through a program. The complexity of a computer program can be correlated with the topological complexity of a graph.

There are various methods of calculating the cyclomatic complexity of a program. Some of them are discussed below.

Method 1:

In mathematical terms, the cyclomatic complexity of a control flow graph G, V(G) can be calculated using the formula given below.
V(G)=E-N+2P
where
E is the number of edges in the control flow graph,
N is the number of nodes in the control flow graph, and
P is the number of connected components (exit points).

Method 2:

Cyclomatic complexity can also be calculated by
V(G)=P+1
where **P** is the number of predicate nodes (nodes containing conditions).

Method 3:

Cyclomatic complexity is the number of regions in a control flow graph.

****

McCabe proposed the cyclomatic number, V (G) of graph theory as an indicator of software complexity. The cyclomatic number is equal to the number of linearly independent paths through a program in its graphs representation. For a program control graph G, cyclomatic number, V (G), is given as:

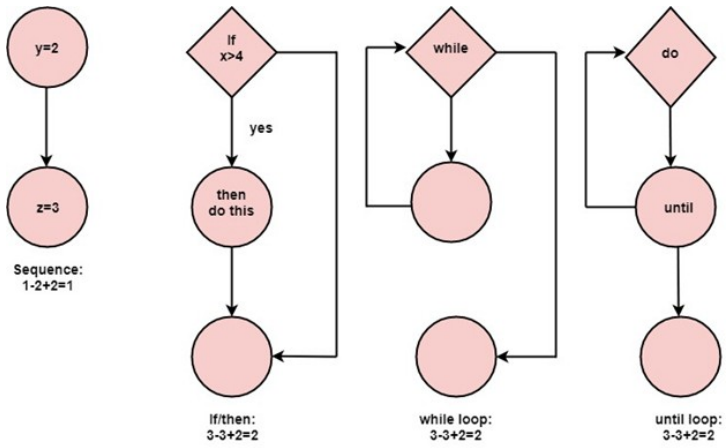
V (G) = E - N + 2 * P

E = The number of edges in graphs G

N = The number of nodes in graphs G

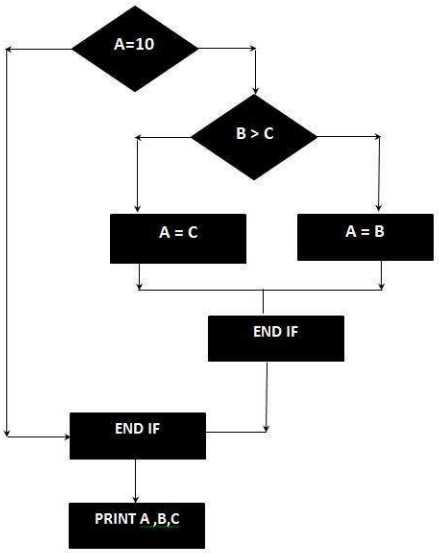
P = The number of connected components in graph G

Example:



Example:

```
IF A = 10 THEN
 IF B > C THEN
   A = B
 ELSE
   A = C
 ENDIF
ENDIF
Print A
Print B
Print C
```



Flow Graph:

seven nodes(shapes)

eight edges (lines), hence the cyclomatic complexity is 8 - 7 + 2 = 3

This translates to the number of decisions + 1.

Cyclomatic Complexity Example

```
void foo(void)
{
   if (a && b)
      x=1;
   else
      x=2;
}
```

At first, it looks like there is one decision in this example. So, CYC = 2.

However, when you consider the side effect of the Boolean operator, there are actually two decisions.

```
void foo(void)
{
   if (a)
      if (b)
         x=1;
   else
      x=2;
 }
```

So, CYC = 3 in this example.

There are other variations of CYC, too.

Myers' Interval is an extension to CYC. It accounts for complexity caused by compound predicates. It uses CYC as its lower bound. The upper bound is defined as the total number of conditions in the code plus 1. It's presented as two values separated by:

CYC: NUMBER OF LOGICAL OPERATORS

```
int divide (int x, int y)
{
   if (y != 0 /* Condition 1 */
   {
   return x / y;
   }
   else if (x == 0 && y > 2) /* Condition 2; Conditional expression 1 */
   {
      return 1;
   }
   else
   {
      printf (“div by zero\n”);
      return 0;
   }
```

The example above has a STMCC value of 3:4. Its CYC is 3. And there is one connective (&&) used in the conditions.

****

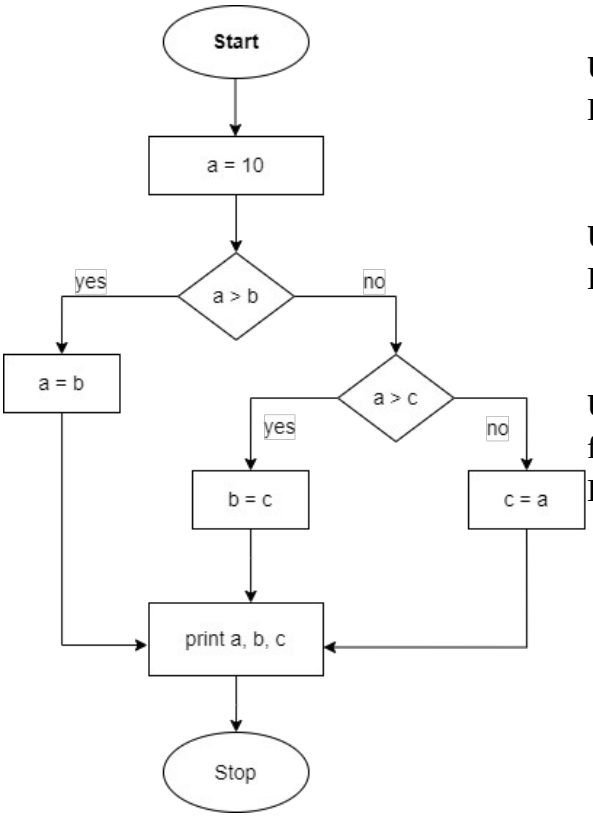
Let us take an **example** to understand the calculation of cyclomatic complexity.

Let us consider the following snippet of code.

```
a = 15 ;
if ( a > b ) then
      a = b ;
else
{
      if ( a > c ) then
            b = c ;
```

```
      else
            c = a ;
      end if;
}
end if;
print a
print b
print c
```

Let us construct the control flow graph now.



Using method 1: E = 10, N = 9, P = 1
Hence, V(G) = E - N + 2*P = 10 - 9 + 2 * 1
V(G) = 3

Using method 2: P = 2
Hence, V(G) = P + 1 = 2 + 1
V(G) = 3

Using method 3: Number of regions in control flow graph = 3
Hence, V(G) = 3

Properties of Cyclomatic complexity:

Following are the properties of Cyclomatic complexity:

1.V (G) is the maximum number of independent paths in the graph

2.V (G) >=1

3.G will have one path if V (G) = 1

4.Minimize complexity to 10

Lower Cyclomatic Complexity = Better Code

Higher numbers of cyclomatic complexity are bad. Lower numbers are good.

That's because code with high complexity is difficult to test. And it's likely to result in errors.

So, code with low complexity is easier to test. And it's less likely to produce errors.